

PROJECT TITLE:Implement Client Script and UI Policy (Incident)

TASK 6: TESTING THE CONFIGURATION

1. Introduction

Testing is an important part of ServiceNow configuration because it verifies whether the implemented UI Policies and Client Scripts are working as expected.

In the previous tasks, a UI Policy and different types of Client Scripts were created for the Incident table. These configurations control field behavior, automatically update values, validate information during submission, and prevent direct list editing of the State field.

Task 6 focuses on testing all the configured behaviors. The testing activities verify whether the Incident form correctly responds to different conditions and user actions.

2. Problem Statement

After implementing UI Policies and Client Scripts, it is necessary to verify that all configurations work correctly.

A configuration may appear correct while still producing unexpected behavior during actual Incident creation or modification. For example, a mandatory field may not be enforced correctly, an Incident may be saved without required information, or a list edit may not be blocked as expected.

Therefore, systematic testing is required to verify each configured behavior and ensure that Incident records follow the defined rules.

3. Objectives

The main objectives of this task are:

- To test the implemented UI Policy.
- To verify mandatory field enforcement.
- To test successful Incident submission.
- To verify automatic Urgency updates.
- To test the Reverse if false behavior.

- To verify that direct State list editing is blocked.
- To verify that State changes through the Incident form are allowed.
- To confirm that all configured Client Scripts work correctly.

4. Skills and Technologies Used

ServiceNow

ServiceNow is used to create and test Incident records.

Incident Management

Incident records are used to verify the configured UI Policy and Client Script behavior.

UI Policy

The UI Policy is tested by changing the Impact value and observing field behavior.

Client Scripts

The configured Client Scripts are tested during field changes, record submission, and list editing.

Form Validation

Form validation is tested by attempting to submit an Incident without the required Assigned To value.

List Editing

List editing is tested to verify that direct State changes are prevented.

5. Testing Overview

The testing process contains five main activities:

1. **Test Mandatory Enforcement**
2. **Test Successful Save**
3. **Reverse Condition Test**
4. **Test List Edit Blocking**

5. Test Form-Based Update

Each activity checks a different part of the configuration.

6. Activity 1: Test Mandatory Enforcement

6.1 Navigate to Incident

Navigate to:

Incident → Create New

This opens a new Incident form.

6.2 Set the Impact Field

Set the **Impact** field to:

High

Changing the Impact to High triggers the configured UI Policy and Client Script behavior.

6.3 Leave Assigned To Empty

Do not enter a value in the **Assigned To** field.

This is done to test whether the onSubmit Client Script prevents the record from being saved.

6.4 Click Submit

Click the **Submit** button to attempt to save the Incident.

6.5 Verify the Result

The Incident should not be saved because the Impact is High and Assigned To is empty.

An error message should appear indicating that the Assigned To field is required for a High-impact Incident.

This verifies the save-validation behavior of the onSubmit Client Script.

The configured UI Policy and other field behaviors should also be checked during this test.

7. Activity 2: Test Successful Save

7.1 Fill the Assigned To Field

Open the Incident form and select a valid user in the **Assigned To** field.

This satisfies the condition required by the onSubmit Client Script.

7.2 Click Submit

After entering the Assigned To value, click **Submit**.

The Incident should now be allowed to save.

7.3 Verify the Result

The Incident should be saved successfully without displaying the previous validation error.

During this test, verify the other configured behaviors as well.

These include:

- Urgency automatically setting to High when required.
- Read-only behavior controlled by the UI Policy.
- State restrictions applied during list editing.

This confirms that the different configurations are working together correctly.

8. Activity 3: Reverse Condition Test

8.1 Open an Existing Incident

Navigate to the Incident records and open an existing Incident where the **Impact** is currently set to High.

8.2 Change Impact

Change the Impact value from:

High → **Medium**

This tests the **Reverse if false** functionality of the UI Policy.

8.3 Verify Field Behavior

After changing the Impact to Medium, verify the following:

- **Assigned To** should no longer be mandatory according to the UI Policy behavior.
- **Urgency** should become editable again because the read-only action is reversed.

This confirms that the UI Policy responds dynamically when its condition is no longer satisfied.

8.4 Save the Record

Click **Submit** to save the changes.

The Incident should save successfully without the High-impact validation condition being applied.

This verifies the reverse-condition behavior.

9. Activity 4: Test List Edit Blocking

9.1 Navigate to Incident List

Navigate to:

Incident → All

The Incident list displays the available Incident records.

9.2 Attempt a List Edit

Locate an Incident record.

Double-click the **State** field to attempt to change it directly from the list.

9.3 Verify the Alert

The onCellEdit Client Script should execute.

An alert message should appear:

State cannot be updated using list editing. Please open the Incident.

The purpose of this message is to inform the user that the State field should be changed through the Incident form.

9.4 Verify the State Value

After the alert appears, the State value should remain unchanged.

This confirms that the onCellEdit Client Script successfully blocks direct State changes from the list view.

10. Activity 5: Test Form-Based Update

10.1 Open the Incident Record

Open the same Incident record used during the previous test.

The purpose is to verify that State changes are still possible through the normal Incident form.

10.2 Change the State

On the Incident form, change the **State** field to another appropriate value.

This tests a form-based update rather than a list-based update.

10.3 Click Update

Click the **Update** button to save the Incident.

10.4 Verify the Result

The State change should be saved successfully.

This confirms that the Client Script is specifically controlling direct list editing rather than preventing all State changes.

The other configured fields should also continue to follow their respective UI Policy and Client Script rules.

11. Overall Testing Results

The testing activities verify the different configurations implemented in the previous tasks.

Test Activity	Expected Result
Mandatory Enforcement	High-impact Incident cannot be saved without Assigned To
Successful Save	Incident saves when Assigned To is provided
Reverse Condition	Field behavior changes when Impact is changed from High
List Edit Blocking	Direct State editing from list is prevented
Form-Based Update	State can be updated through the Incident form

These tests provide a complete verification of the configured Incident behavior.

12. Testing Benefits

Testing provides several important benefits.

It confirms that the implemented configurations work according to their intended conditions.

It also helps identify configuration problems before the solution is used in normal Incident management.

Testing the mandatory field behavior ensures that incomplete High-impact Incidents cannot be submitted.

Testing the reverse condition verifies that the UI Policy does not unnecessarily restrict fields when its condition is false.

Testing list editing confirms that direct State changes are controlled while form-based updates remain available.

13. Expected Overall Result

After completing all testing activities, the following behaviors should be confirmed:

- High-impact Incidents require the appropriate information.
- Assigned To validation works during submission.
- Urgency is automatically updated according to the configured Client Script.
- Urgency becomes read-only under the High Impact UI Policy.
- UI Policy behavior is reversed when the condition is no longer true.
- Direct State changes from the Incident list are blocked.
- State changes through the Incident form are allowed.

14. Result

All major configurations implemented in the **Implement Client Script and UI Policy (Incident)** project were tested through different Incident scenarios.

The testing activities verify mandatory field enforcement, successful record submission, reverse UI Policy behavior, list editing restrictions, and form-based updates.

15. Conclusion

Task 6 successfully demonstrates the testing and verification of the UI Policy and Client Script configurations implemented for the Incident table.

The testing process confirms that High-impact Incidents are properly validated, required information is enforced, and configured field behavior changes according to the Impact value.

The testing also confirms that direct State changes through list editing are prevented while State changes through the Incident form are allowed.

Overall, this task completes the project by verifying that the implemented ServiceNow configurations perform their intended functions and provide controlled, consistent behavior for Incident records.